

CRA TEAM 3 · CTRL-C

CodeFab

app 패키지로 정리된 Custom Language Interpreter에 `ctrl_c()`,
`explain`, 한글 키워드를 더한 팀 프로젝트입니다.

Custom Language

Interpreter Pipeline

Code Review

GitHub Actions

Design Patterns

GOAL

우리가 만든 것

언어

- 변수, 조건문, 반복문, 함수
- 클래스, 메서드, 생성자, 상속
- 배열, 정적 검사, line-aware error
- `ctrl_c()`, `explain`, 한글 키워드

실행기

- `app/` 패키지 기준 구조 정리
- 소스 코드를 Token, AST로 변환
- 실행 전 Checker로 오류 탐지
- Executor가 AST를 순회하며 실행

개발 문화

- PR template 기반 리뷰
- 최소 2명 approve
- GitHub Actions로 lint/test 자동화
- Makefile로 로컬 명령 통일

LANGUAGE PIPELINE

CodeFab은 공장처럼 동작합니다

Tokenizer

문자열을 의미 있는 부품인 Token으로 자릅니다.

Assembler

Token을 조립해서 Expr/Stmt 트리로 만듭니다.

Checker

실행 전에 초기화, scope, 호출 오류를 확인합니다.

Executor

AST를 순회하며 실제 결과를 만듭니다.

최신 main 기준으로 `app/assembler`, `app/checker`, `app/executor`, `app/shell` 이 같은 파이프라인을 공유합니다.

DEMO SCRIPT

기본 기능 실행 예시

```
Func sum(n) {  
    var total = 0;  
  
    for (var i = 0; i <= n; i = i + 1) {  
        total = total + i;  
    }  
  
    return total;  
}  
  
print sum(5);
```

```
$ make run demo.cfab  
15
```

CLASS DEMO

클래스와 상속

```
Class Robot {  
    move() {  
        print "robot";  
    }  
}  
  
Class SpeedRobot : Robot {  
    move() {  
        Super.move();  
        print "speed";  
    }  
}  
  
var r = SpeedRobot();  
r.move();
```

동작 흐름

- **Class** 는 호출되면 instance를 만듭니다.
- **This** 는 현재 instance를 가리킵니다.
- **Super** 는 부모 클래스의 메서드를 찾습니다.
- Checker는 잘못된 위치의 **Super** 를 잡습니다.

작지만 CodeFab다운 확장

`ctrl_c()`

- CodeFab 코드 안에서 호출하는 추천/보조 함수입니다.
- 현재 코드와 실행 맥락을 바탕으로 다음 행동을 돕습니다.
- Issue #12의 `ctrl c` 아이디어를 언어 기능으로 옮겼습니다.

`explain`

- 입력 코드가 파이프라인을 어떻게 지나는지 보여줍니다.
- Debug Mode와 다르게 실행을 멈추지 않습니다.
- 구조 설명을 실제 출력으로 보여줄 수 있습니다.

한글 키워드 별칭

- 기존 문법을 새로 만들지 않습니다.
- Tokenizer 단계에서 표준 Token으로 정규화합니다.
- 내부 AST, Checker, Executor는 그대로 유지합니다.

EXPLAIN

구조를 보여주는 기능

```
explain {  
  var x = 1 + 2 * 3;  
  ctrl_c();  
}
```

```
[Tokens]  
VAR IDENTIFIER EQUAL NUMBER PLUS NUMBER STAR NUMBER SEMICOLON  
IDENTIFIER LEFT_PAREN RIGHT_PAREN SEMICOLON EOF
```

```
[AST]  
BlockStmt(  
  VarStmt(x, BinaryExpr(1, +, BinaryExpr(2, *, 3))),  
  ExpressionStmt(CallExpr(ctrl_c))  
)
```

```
[Checker]  
No errors
```

```
[Result]  
x = 7  
ctrl_c: next step suggested
```

한글 키워드는 기존 문법의 별칭입니다

기존	한글 alias	의미
<code>print</code> , <code>var</code>	출력, 값	값을 이름에 묶습니다.
<code>if</code> , <code>else</code> , <code>for</code>	만약, 아니면, 반복	흐름 제어를 표현합니다.
<code>Func</code> , <code>return</code>	함수, 반환	함수를 선언하고 결과를 돌려줍니다.
<code>Class</code> , <code>init</code>	클래스, 초기화	객체의 형태와 생성 시 동작을 정의합니다.
<code>This</code> , <code>Super</code>	이것, 부모	현재 instance와 부모 class를 가리킵니다.
<code>true</code> , <code>false</code>	참, 거짓, ○○, LL	Boolean literal입니다.
<code>and</code> , <code>or</code>	그리고, 또는	조건을 조합합니다.

한글 alias 실행 예시

```
함수 합계(n) {  
    값 total = 0;  
  
    반복 (값 i = 0; i <= n; i = i + 1) {  
        total = total + i;  
    }  
  
    반환 total;  
}  
  
값 enabled = 참;  
값 result = 합계(5);  
  
만약 (enabled 그리고 result > 10) {  
    출력 "large";  
} 아니면 {  
    출력 "small";  
}
```

Composite Pattern: 코드를 트리로 다루기

교안의 예시

- `3 + 2 * 7` 을 계산 트리로 표현합니다.
- 숫자도 Expression, 연산도 Expression입니다.
- 부분과 전체를 같은 방식으로 다룹니다.

CodeFab의 적용

- `LiteralExpr`, `BinaryExpr` 는 모두 `Expr` 입니다.
- `PrintStmt`, `BlockStmt` 는 모두 `Stmt` 입니다.
- `Executor`는 AST 트리를 내려가며 값을 계산합니다.

```
print 1 + 2 * 3;
```

```
PrintStmt
```

```
├ BinaryExpr(+)
│   ├── LiteralExpr(1)
│   └ BinaryExpr(*)
│       ├── LiteralExpr(2)
│       └ LiteralExpr(3)
```

Adapter Pattern: 다른 호출 대상을 같은 규칙으로

문제

- 내장 함수, 사용자 함수, 클래스 생성자는 내부 구현이 다릅니다.
- 하지만 언어 사용자에게는 모두 "호출"입니다.

해결

- `Callable` 이라는 공통 호출 규칙을 둡니다.
- `NativeFunction`, `UserFunction`, `UserClass` 가 `call()` 을 제공합니다.
- `Executor`는 세부 구현을 몰라도 됩니다.

```
add(1, 2);      // NativeFunction
fact(5);        // UserFunction
Robot("A");     // UserClass
```

```
Executor -> callee.call(executor, arguments)
```

Factory-like: 생성 책임을 한 곳으로 모으기

Tokenizer

- `"var"` 를 `Token(VAR)` 로 만듭니다.
- `"123"` 을 `Token(NUMBER)` 로 만듭니다.
- 한글 alias도 이 단계에서 정규화합니다.

Assembler

- `var x = 1;` 을 `VarStmt` 로 만듭니다.
- `x + 1` 을 `BinaryExpr` 로 만듭니다.
- 새 문법의 생성 지점을 찾기 쉽습니다.

정통 Factory Method라기보다는, 객체 생성 책임을 모은 Factory 스타일 구조로 설명하는 것이 정확합니다.

DESIGN PATTERN 4

Command Pattern: Custom 기능을 명령으로 분리하기

Before

```
if name == "ctrl_c":  
    suggest()  
if keyword == "explain":  
    print_pipeline()  
if keyword in korean_alias:  
    normalize()
```

리팩토링 후보

```
CtrlCCommand  
ExplainCommand  
AliasNormalizeCommand  
RunProgramCommand
```

```
Interpreter -> command.execute(context)
```

custom 기능이 늘어날수록 조건문을 키우기보다, 기능별 명령 객체로 분리하는 편이 테스트와 설명에 유리합니다.

DESIGN CHOICE

Singleton은 일부러 쓰지 않았습니다

- Prompt Shell은 세션 동안 상태를 유지해야 합니다.
- 하지만 테스트는 매번 깨끗한 실행 환경이 필요합니다.
- `Executor` 는 각 인스턴스마다 `globals` , `environment` , `outputs` 를 가집니다.
- 패턴은 무조건 적용하는 것이 아니라, 필요한 상황에만 선택했습니다.

REVIEW CULTURE

리뷰로 기능을 완성했습니다

PR 방식

- PR template 작성
- 최소 2명 approve
- 작성자가 직접 merge
- 작은 기능 단위 branch 사용

리뷰에서 잡은 문제

- 중첩 statement의 runtime error line
- Super 사용 위치 checker 누락
- 알 수 없는 statement가 조용히 무시되는 문제
- class/array 정적 검사 보강

좋았던 점

- 기능 구현과 검증 기준을 PR에 함께 기록
- 댓글로 재현 예시를 남김
- 리뷰가 단순 승인보다 품질 개선으로 이어짐

AUTOMATION

Makefile과 GitHub Actions

로컬 명령

- `make install`: 개발 의존성 설치
- `make lint`: black, isort, ruff 실행
- `make test`: pytest 전체 실행
- `make run <file>`: 파일 모드 실행
- `make debug <file>`: 디버그 모드 실행
- 로컬에서도 CI와 같은 기준을 사용합니다.

CI

- Lint workflow: formatting, import order, ruff
- Unit Tests workflow: pytest와 coverage
- Gist Acceptance workflow: 외부 예제 기반 수용 테스트
- `app` 패키지 기준 coverage 확인